المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

M127_1 Compilation

Dr Oussama SMIMITE
o.smimite@uiz.ac.ma

Objectifs du cours

- *Principes et les techniques relatifs aux traducteurs de langages de Programmation ainsi que les outils de développement des traducteurs .*
- *Notions de traducteur (compilateur)*
- *Génération de code.*
- *Optimisation du code.*
- *Écrire des analyseurs lexicaux .*
- *Écrire des analyseurs syntaxiques .*
- *Réaliser des traducteurs .*

Séance 1

Introduction

Objectif: initiation à la théorie des langages formels

Langues (langages): supports de communication

- Langages: permettent de communiquer avec les machines.

- Langues: permettent aux hommes d'échanger des informations et des idées

un langage de programmation: est le moyen privilégié dont dispose le programmeur pour communiquer ses algorithmes à l'ordinateur.

idéal = communication à l'aide d'un langage naturel …

Problème: les langues naturelles sont intrinsèquement ***ambiguës***, le sens d'une phrase dépend souvent du contexte dans lequel la phrase est utilisée.

Un langage de programmation devra être non ambigu si nous voulons que la machine exécute correctement nos ordres.

C’est des langages crées artificiellement

c'est la théorie des langages formels.

Historique

Avant 1950

- Langage machine : suite de nombres écrits dans le système hexadécimal. Certains nombres représentent des instructions, d'autres des valeurs, etc. (0A 43)
- Langage d'assemblage : il contient des instructions faisant intervenir les registres de l'unité centrale de l'ordinateur, les données.

  Ex : ADD #2, R1
- Introduction des étiquettes dans le langage d'assemblage dans le but de faciliter les branchements (boucles, sauts).
- Introduction des variables Ex : MOV R1,b

Définition des Langages de haut niveau (1950)

- expressions
- Fonctions
- Structures de données

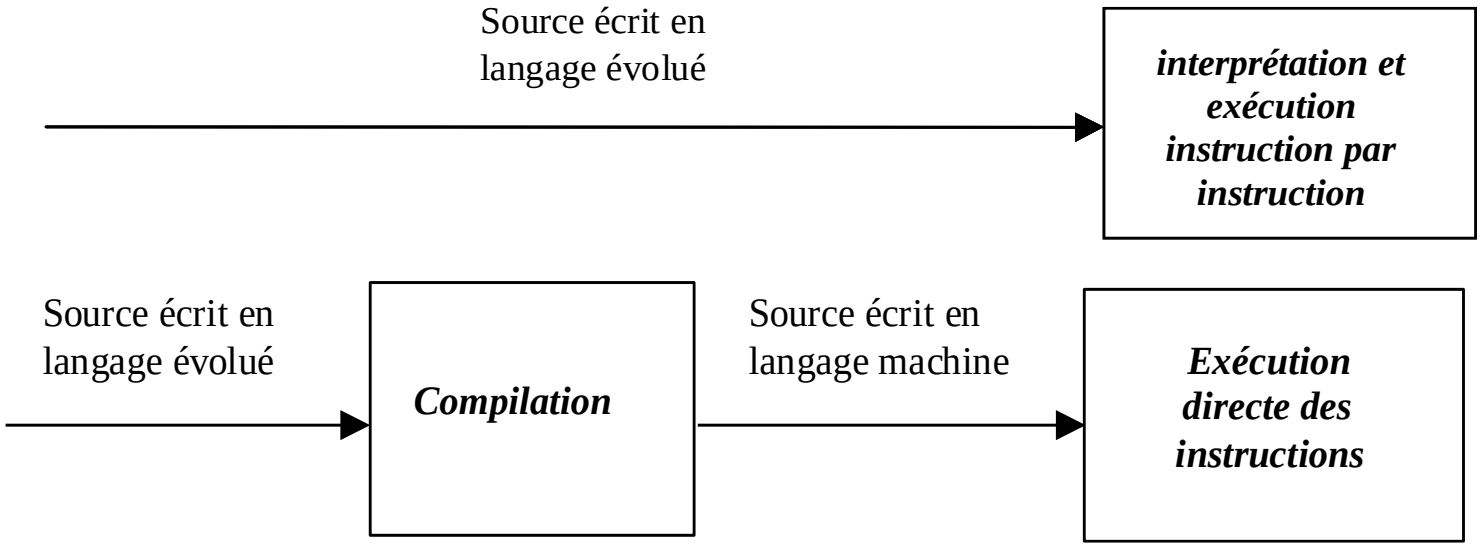
Définition des langages à objets (1970)

- Classes, Objets, méthodes, variables d'instance
- Héritage /polymorphisme

- En 1957 Chomsky: défini des langages à partir des grammaires génératives.
- Les langages (comme pour les langues naturelles) ont:
  - un **alphabet**,
  - des **mots**
  - et une **grammaire**.
- Un langage source bien écrit doit donc être fait de phrases formées de mots du langage dont l'assemblage doit respecter des règles de grammaire.

Un ordinateur ne comprend que son langage machine.
Il est nécessaire d'utiliser un interprète ou bien un compilateur.

Pour communiquer avec des langages plus évolués



Il y a toujours une étape d'analyse de source pour pouvoir traduire une instruction en langage machine, il faut que l'instruction en langage évolué soit comprise.

Compilateur

Tout programme écrit dans un langage de haut niveau doit être traduit en instructions exécutables par un ordinateur. Le programme effectuant cette traduction est **un compilateur.**

Il est préférable que le programme soit indépendant du système sur lequel il doit s'exécuter.

Le programme n'a pas besoin du compilateur lorsqu'il est exécuté.

La compilation est un peu plus compliquée.

Exemples de langages compilées : C, C++,...

Interpréteur

Un interpréteur travaille simultanément sur les données et sur le programme source.

Il analyse une à une les instructions du programme, les “compile” et les exécute.

L’interpréteur doit être présent sur le système où le programme s’exécute.

L’exécution est plus lente.

On peut modifier le source pendant l’exécution .

Exemples de langages interprétés : PHP, JavaScript….

Langage Intermédiaire

Le code source est traduit dans une forme binaire (pseudo-code).

Ce code est interprété et exécuté.

Exemple de Java :

fichier source (.java) → compilateur (javac) → fichier objet (.class)

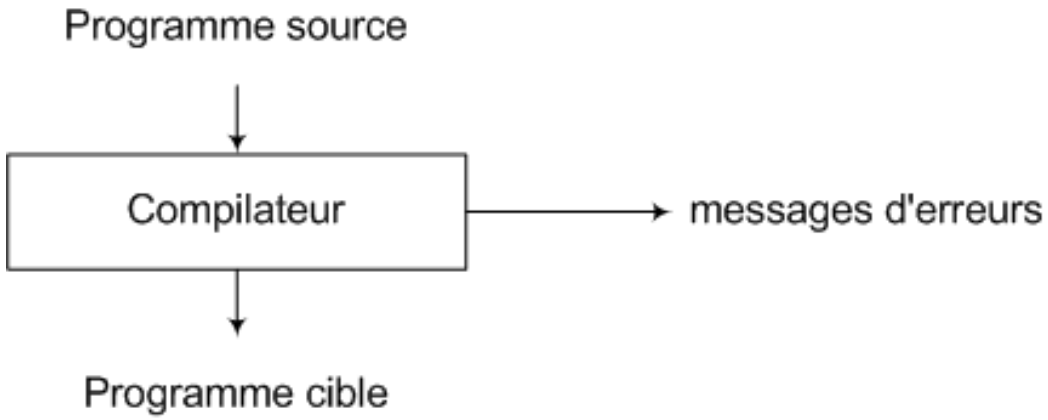
Les fichiers objet (et les ressources) peuvent être regroupés dans des fichiers archives (.jar).

Les fichiers objet → interpréteur (JVM) → résultat.

Exemples de langages intermédiaires : Java, Python.

Phases de la compilation

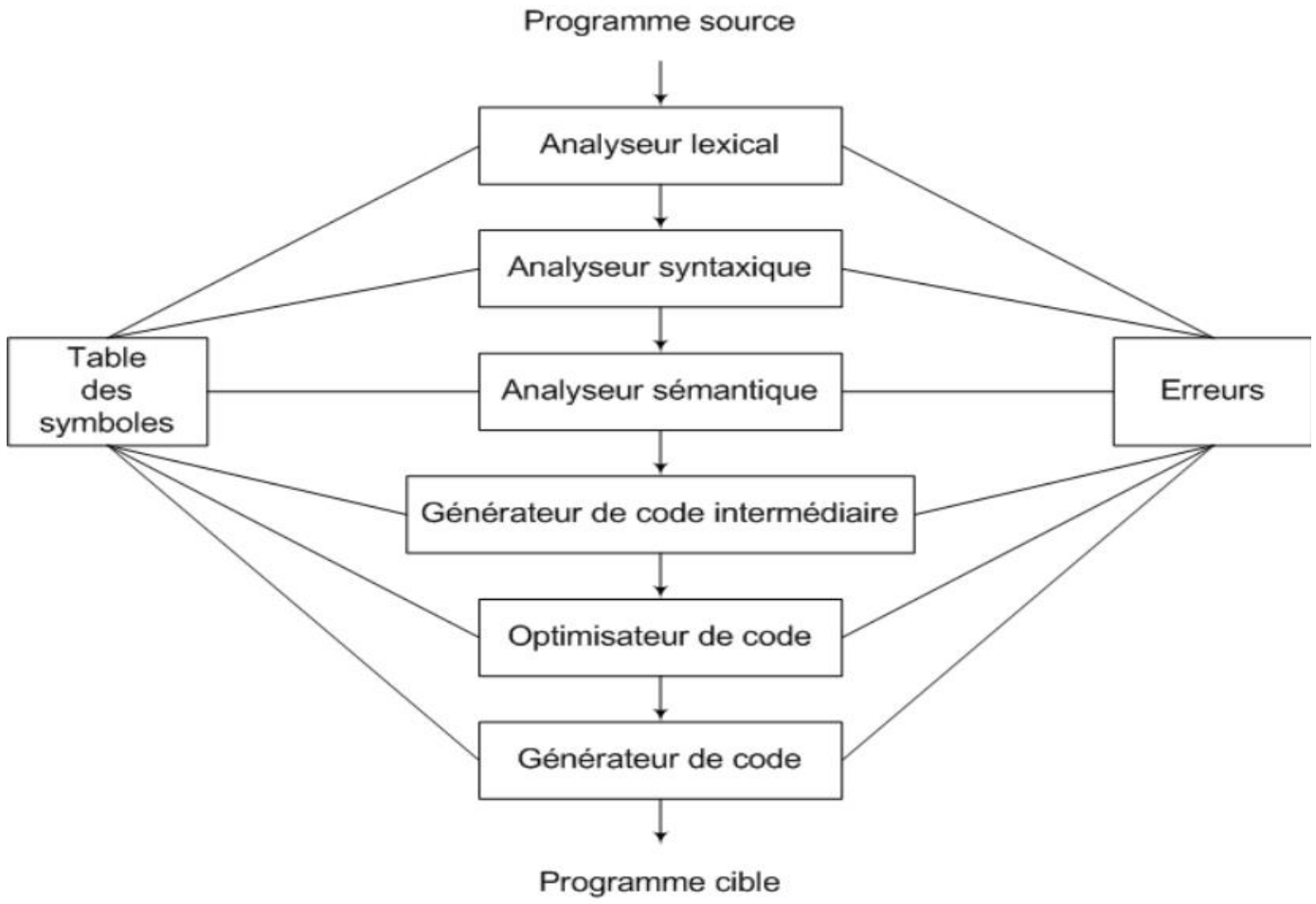
Le but d'un compilateur est de traduire un programme source écrit dans un certain langage en un programme cible écrit dans un dans un autre langage. Le plus souvent, le langage cible est le langage machine, mais cela peut être seulement un code intermédiaire optimisé.



La différence entre un interpréteur et un compilateur

Interpréteur	Compilateur
Convertit le programme en prenant une seule ligne à la fois.	Analyse l'ensemble du programme et le traduit dans son ensemble en code machine.
L'analyse du code source prend moins de temps, mais le temps d'exécution global est plus lent.	L'analyse du code source prend beaucoup de temps, mais le temps d'exécution global est comparativement plus rapide.
Aucun code d'objet intermédiaire n'est généré, la mémoire est donc efficace.	Génère du code d'objet intermédiaire qui nécessite en outre une liaison, nécessite donc davantage de mémoire.
Continue de traduire le programme jusqu'à ce que la première erreur soit rencontrée. Par conséquent, le débogage est facile.	Il génère le message d'erreur uniquement après avoir analysé l'ensemble du programme. Par conséquent, le débogage est relativement difficile.

La compilation se décompose en 6 phases : 3 phases d'analyse et 3 phases de génération de code.



Analyse du programme source

L'analyse du programme source est réalisée durant les trois premières phases de la compilation.

- **Analyse lexicale**. Le flot des caractères du programme source est regroupé en suites ayant une signification précise et appartenant à des catégories prédéfinies appelées unités lexicales.
- **Analyse syntaxique**. Elle regroupe les unités lexicales en structures ou unités grammaticales qui permettent de synthétiser le résultat de la compilation.
- **Analyse sémantique**. Elle contrôle la cohérence sémantique du programme source. En particulier, elle vérifie la cohérence des types des variables et des expressions et procède parfois à des conversions (entier vers réel).

La représentation interne du programme source évolue en fonction des phases de la compilation.

Analyse du programme source

Exemple:

aire = base * hauteur / 2

Analyse lexicale

L'analyse lexicale reconnaît les lexèmes suivants :

aire	=	base	*	hauteur	/	2

aire, base, hauteur sont des lexèmes représentants de la même unité lexicale appelée :**identificateur.**

La représentation de l'instruction peut être suggérée par :

id1 = id2 ∗ id3/2

Analyse du programme source

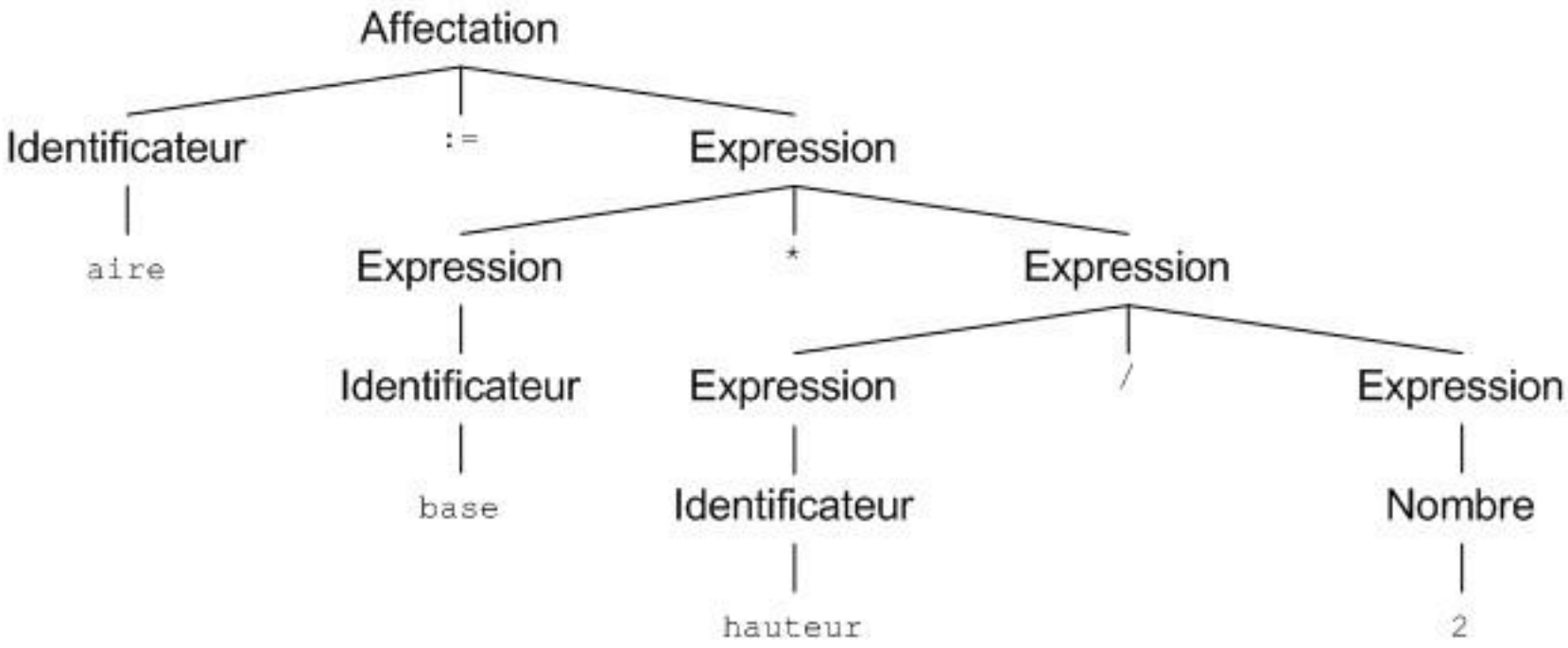
Apres analyse lexicale, la chaîne qui contient l'instruction source :
aire = base * hauteur / 2 peut être représentée par la suite :
<identificateur> aire
<affectation> =
<identificateur> base
<operateur> *
<identificateur> hauteur
<operateur> /
<nombre> 2

Analyse du programme source

Analyse syntaxique

L'analyse syntaxique regroupe les unités lexicales en structures grammaticales en suivant les règles figurant dans une grammaire.

On représente souvent par un arbre syntaxique le résultat produit par l'analyse syntaxique.



Analyse du programme source

Analyse syntaxique

La structure hiérarchique d'un programme est exprimée à l'aide de règles. Ainsi :

Tout identificateur est une expression.

Tout nombre est une expression.

Certaines règles sont récursives :

Si expr1 et expr2 sont des expressions alors expr1 $*$ expr2 est une expression

Analyse du programme source

Analyse sémantique

L'analyse sémantique va contrôler les erreurs sémantique et collecte des informations de type destinées aux phases suivantes de la compilation.

Une partie importante est le contrôle des types des variables (identificateurs de variables : aire, base, hauteur) dans les instructions.

- Affectation
- Opérandes dans les opérations, facteurs dans les expressions
- Type de retour des fonctions

Les erreurs de types sont généralement signalées et nécessitent une correction .

Ex : nombre réel employé comme indice d'un tableau.

Analyse du programme source

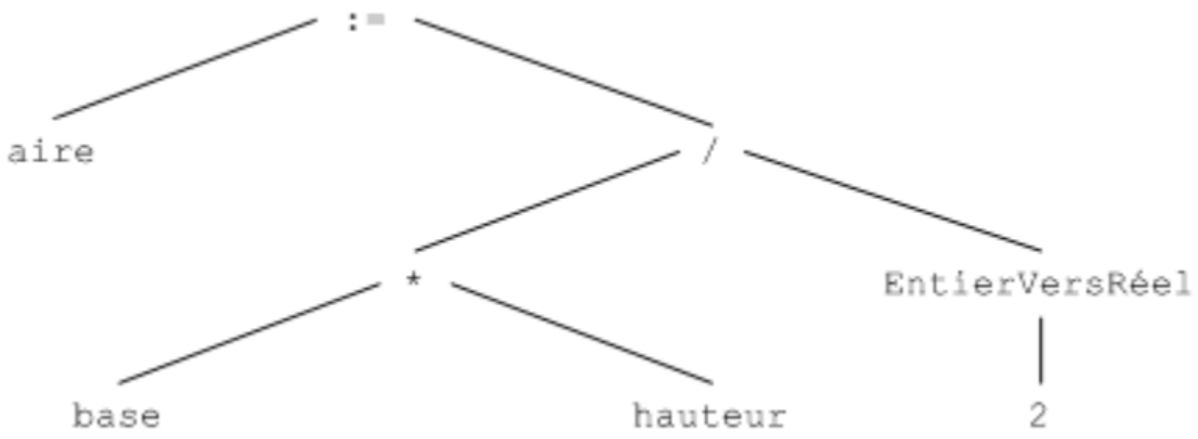
Cependant, l'analyse sémantique peut éventuellement réaliser la coercition de valeurs ou de constantes.

Dans l'instruction étudiée, les identificateurs base, hauteur sont normalement déclares comme réels.

Les opérations, multiplication, division ont des opérandes de même type (tous deux réels).

L'entier 2 va donc être substitue par un réel (2.0). Le codage machine des entiers et des réels n'´étant pas le même cette coercition est rendue nécessaire.

La figure montre l'appel à un operateur supplémentaire.

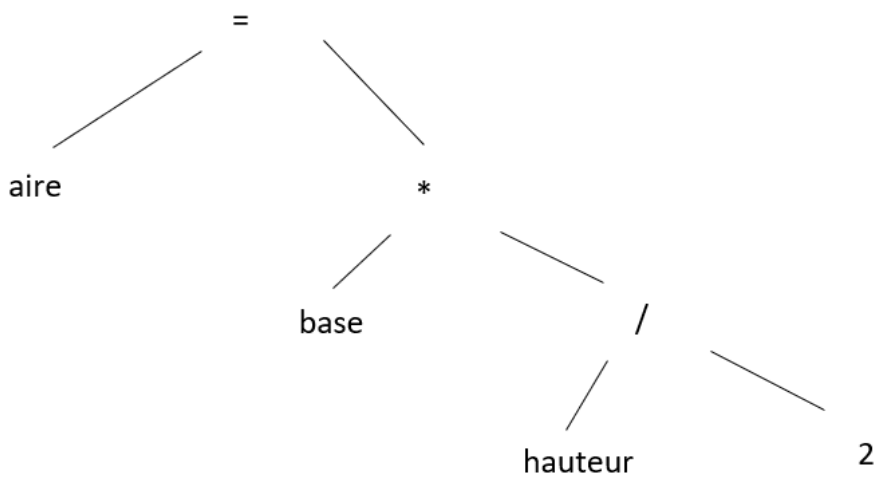


Génération de code intermédiaire

- Arbre abstrait (une représentation hiérarchique d’une expression)
- Code à trois adresses

Génération de code intermédiaire

Arbre abstrait



écriture en notation préfixe

=(aire,*(base,/(hauteur,2)))

Génération de code intermédiaire

Code à trois adresses

Le code a trois adresses est une version simplifiée de l'assembleur ou Il s'agit simplement de "découper" un code simple en morceaux de taille trois : deux arguments, un résultat.

a := b + f(2*c);

devient

***t*0 := 2 ∗ c**

***t*1 := f(*t*0)**

a := b + *t*1

Optimisation de code

Code initial

```
a=1;
b=2+a;
c=b+3;
d=c;
```

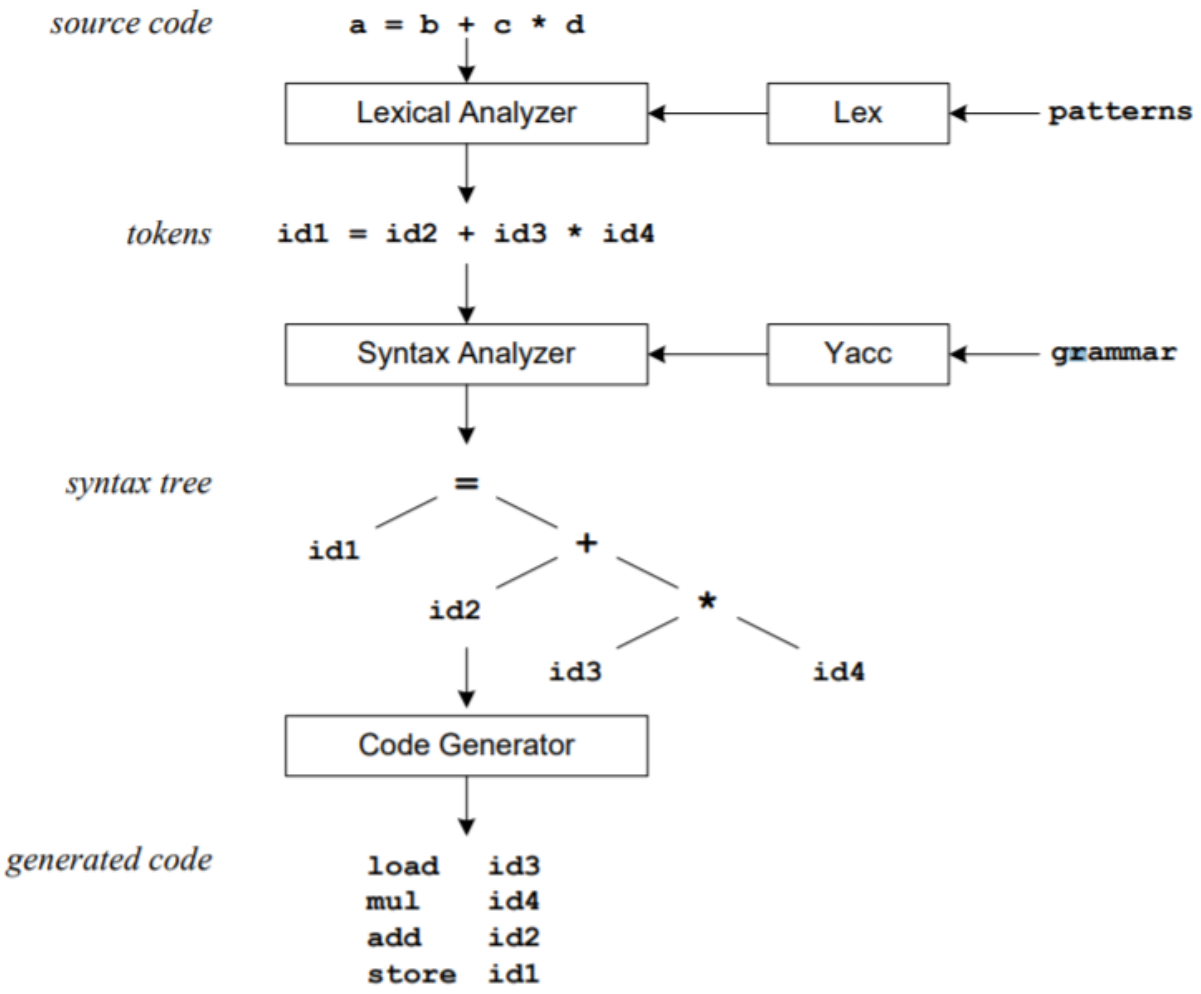
Optimisation

```
b=2+1;
d=b+3;
```

Analyse en trois étapes:

- *Analyse lexicale:* permet de décomposer le texte en entités élémentaires appelées lexèmes (token en anglais).
- *Analyse syntaxique:* permet de reconnaître des combinaisons de lexèmes formant des entités syntaxiques.
- *Analyse sémantique* permet de gérer le typage et générer le code objet directement compréhensible par la machine (ou bien un code intermédiaire qui devra être de nouveau traduit dans un code machine).

Outils d'analyse



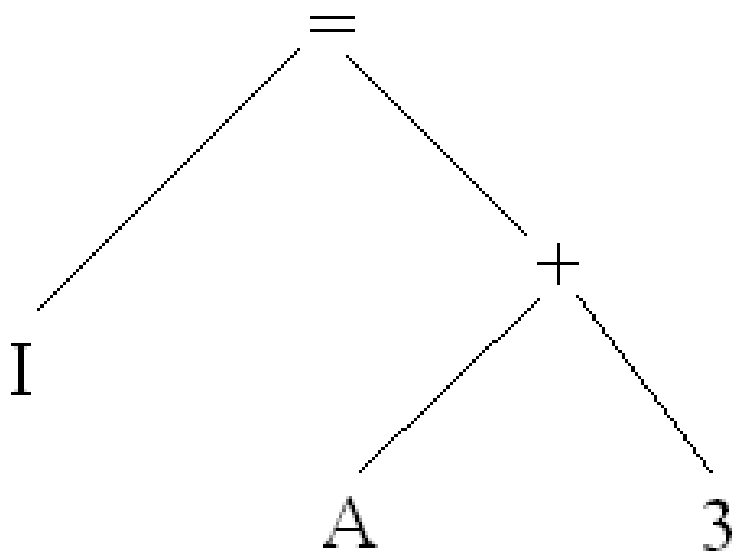
Exemple:

- I = A + 3 ; (**I:** entier, **A:** réel):

- **Analyse lexicale:**
- **I : identificateur 1 entier**
- **= : opérateur d'affectation**
- **A : identificateur 2 réel**
- **+ : opérateur arithmétique**
- **3 : constante "entier"**
- **; : point virgule**

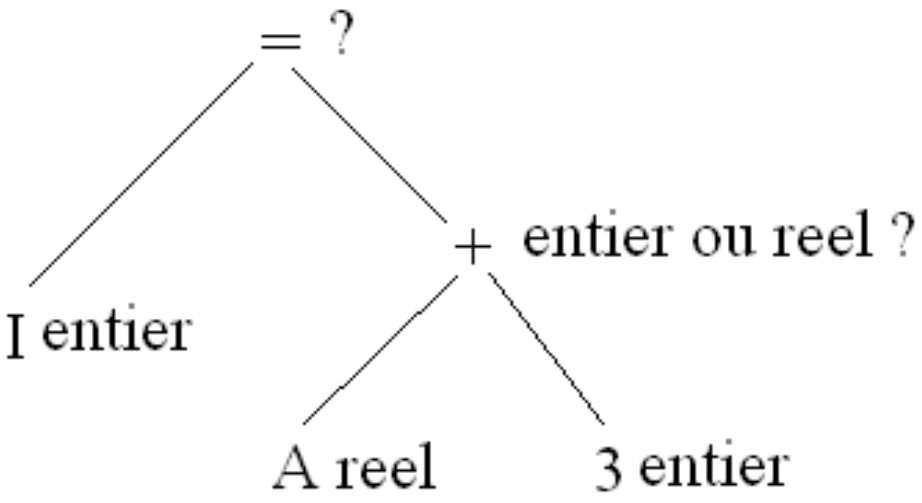
Analyse syntaxique

- Permet de reconnaître que cette combinaison de lexèmes forme une instruction C.



Analyse sémantique:

- vérifie le bon typage des **opérandes**,
- génère le code objet correspondant à cette instruction.



- Il y a plusieurs points communs entre l'analyse lexicale et l'analyse syntaxique.
- Cette partie commune aux deux processus est la nature même de la **théorie des langages**.
- **On trouvera donc dans cette théorie:**
- Les concepts de langage et de grammaire,
- La catégorisation des grammaires et par la même des langages,
- Les propriétés des différentes catégories de langages et de grammaires,
- Les concepts algorithmiques permettant de chercher et/ou valider les mots et les phrases d'un langage (les différents types d'automates).

Exercices d'application

Donner l'arbre abstrait (schéma et notation préfixe) des expressions suivantes:

- a+b+1
- a*b+1
- 2*(x+1)

Donner le code à trois adresses de

- a=b+3*c+1;
- d=h*2+d*3

Rappel Mathématiques

Ensemble = collection d'objets

- Il sont désignés par des lettres majuscules A, B, W
- L'ensemble vide, et est noté {} ou bien Ø.
- **Exemple:**
- l'ensemble des chiffres en base 10 est {0,1,2,3,4,5,6,7,8, 9}
- $\{x \mid P(x)\}$ qui désigne l'ensemble contenant tous les objets x pour lesquels la condition P est vraie.
- $\{x \mid x$ est un entier pair$\}$ désigne l'ensemble des entiers pairs
- $\{x \mid x$ a une moyenne > 10$\}$
- $\{x \in A \mid P(x)\}$ comme $\{x \in \mathbb{Z} \mid x$ est un entier pair$\}$
- $\{F(x) \mid x \in A\}$ comme $\{2x \mid x \in \mathbb{Z}\}$
- $\{F(x) \mid P(x)\}$ comme {propriétaire de x | x est un chien}

Opérations sur les ensembles

Soient E et F deux ensembles de $\mathbb{R}$, nous définissons les différentes opérations ensemblistes sur E et F comme suit :

L'inclusion : $E \subset F : \forall x \in E \Rightarrow x \in F$

L'union U : $E \cup F = \{x \in \ / x \in E \text{ ou } x \in F\}$

L'intersection : $E \cap F = \{x \in / x \in E \text{ et } x \in F\}$

Le complémentaire de F dans E (avec $F \subset E$) : $C_E F = \{x \in \mathbb{R} / x \in E \text{ et } x \notin F\}$

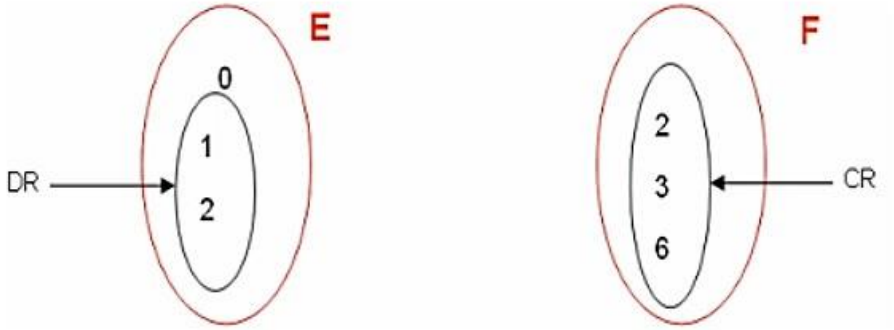
Le produit cartésien : $E X F = \{(x,y) \in \mathbb{R} / x \in E \text{ et } y \in F\}$

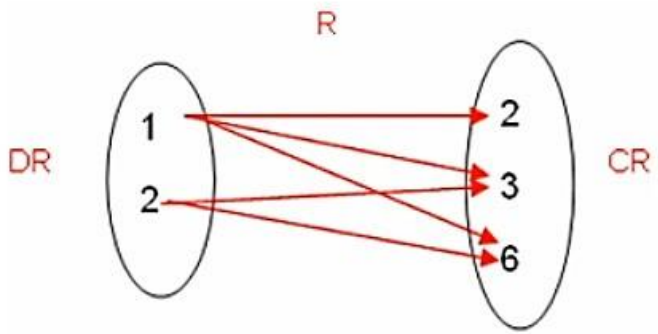
Soient E et F deux ensembles

- Une relation R de E dans F est un sous ensemble des couples (x, y) du E x F tel que x R y.
- Une relation R sur un ensemble E est un sous ensemble des couples (x, y) du produit cartésien E x E tel que x R y.

Exemple

Soit R est définie comme suit : x est strictement inférieur à y et x ≠ 0. Le





- ExF = {(1,2), (1,3), (1,6), (2,2), (2,3), (2,6), (0,2), (0,3),(0,6)}
- Le sous ensemble du produit cartésien qui vérifie cette relation est {(1,2), (1,3), (1,6), (2,3), (2,6)}.

Relation d'équivalence

Définition

On dit qu'une relation R est une relation d'équivalence si elle est :

- symétrique : $\forall x \in E,\ \forall y \in E,\ xRy \Rightarrow yRx,$
- réflexive : $\forall x \in E,\ xRx,$
- transitive : $\forall x \in E,\ \forall y \in E,\ \forall z \in E,\ (xRy \text{ et } yRz) \Rightarrow xRz.$

Dans le cas d'une relation d'équivalence, deux éléments en relation sont aussi dits équivalents.

Relation d'équivalence

Exemples:

1. La relation R « être parallèle » est une relation d'équivalence pour l'ensemble E des droites affines du plan :
   - réflexivité : une droite est parallèle à elle-même,
   - symétrie : si D1 est parallèle à D2 alors D2 est parallèle à D1,
   - transitivité : si D1 parallèle à D2 et D2 parallèle à D3 alors D1 est parallèle à D3.
2. La relation « être du même âge » est une relation d'équivalence.
3. La relation « être perpendiculaire » n'est pas une relation d'équivalence (ni la réflexivité, ni la transitivité ne sont vérifiées).

Relation d'équivalence

Classe d'équivalence:

Soit R une relation d'équivalence sur un ensemble E. Soit $x \in E$, la classe d'équivalence de x est l'ensemble des éléments qui sont en relation avec x.
On note cette ensemble par : $cl(x) = \{y \in E \mid y \, R \, x\}$.

Exemple

La relation « être du même âge » est une relation d'équivalence. Si on prend une personne appelée Ali et âgée de 20 ans. Donc la classe d'équivalence de Ali par cette relation est l'ensemble des personnes qui ont le même âge que lui.

Relation d'ordre

Définition:

Soit E un ensemble. Une relation binaire R dans E est une relation d'ordre si elle est :

- Réflexive : $\forall x \in E,\ xRx$
- Antisymétrique : $\forall x \in E,\ \forall y \in E,\ (xRy \text{ et } yRx) \Rightarrow x = y$
- Transitive : $\forall x \in E,\ \forall y \in E,\ \forall z \in E,\ (xRy \text{ et } yRz) \Rightarrow xRz$

Exemple

- La relation d'ordre $\leq$ dans l'ensemble des réels.
- La relation d'inclusion sur $\subset$ les parties d'un ensemble.
- La relation $x < y$ sur $\mathbb{R}$ n'est pas une relation d'ordre car elle n'est pas réflexive.

Relation d'ordre total

Définition:

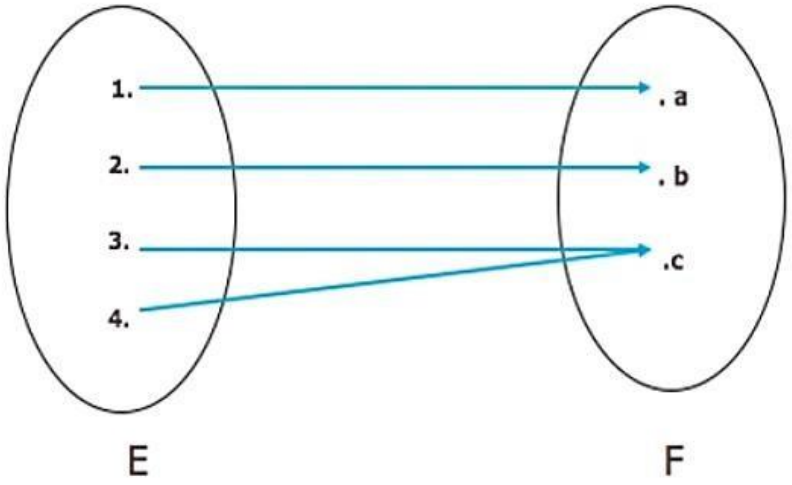
On dit qu'une relation R dans un ensemble E est une relation d'ordre total si tous les éléments de E sont comparables par cette relation.

Exemple

- Etant donnés deux nombres réels x et y , on a toujours $x \leq y$ ou $y \leq x$:
- La relation d'inclusion entre sous-ensembles d'un ensemble E n'est pas une relation d'ordre total sur P(E). Il existe des ensembles tel que le premier ne soit pas inclus dans le second, ni le second inclus dans le premier. Par exemple $[1,3] \not\subset [0,2]$ et $[0,2] \not\subset [1,3]$ pour des intervalles de $\mathbb{R}$.

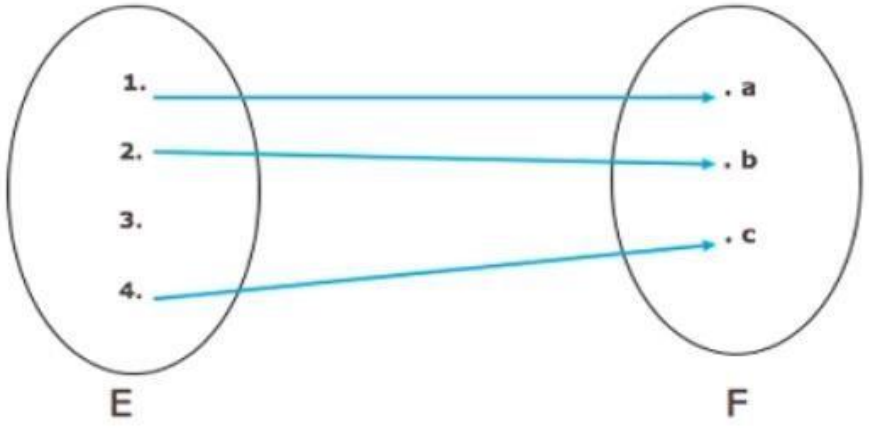
Application

La relation définit dans le schéma ci-dessous est une application car tout élément de E a une image dans F.



Fonction

La relation définit dans le schéma ci-dessous est une fonction car tout élément de E a au plus une image dans F.



Exemple : f : x $\longrightarrow$ 1/x est une fonction de $\mathbb{R}$ dans $\mathbb{R}$ mais pas une application car « 0 » n'a pas d'image.

Loi de composition interne

Une loi de composition interne sur un ensemble E est une application qui associe à chaque couple $x \in E$, et $y \in E$ un élément $x*y \in E$.

$$(*) : E \times E \longrightarrow E$$
$$(x, y) \longrightarrow x*y$$

Si on note cette loi par (*), alors on écrit :

1. (*) est commutative si: $\forall\, x, y \in E \quad x*y=y*x$
2. (*) est associative si : $\forall\, x, y,$ et $z \in E \; x*(y*z)=(x*y)*z=x*y*z$.
3. (*) a un élément neutre « e » si: $\forall\, x \in E \; x*e=e*x=x$

Monoïde

Un monoïde E est un ensemble muni d'une loi de composition interne * Associative et possédant un élément neutre « e ». On écrit : < E, *, e >.

Exemple :

< IN,+,0 >
< IN, x ,1>

Morphisme

Soient E et F deux ensembles munis respectivement des lois de composition internes « * » et « . »

On dit que la fonction f est un morphisme de E dans F si et seulement si

$$f(x * y) = f(x).f(y)$$

Soient < E, *, e > et < F, . , e'> deux Monoïdes ou «e», «e'» sont les éléments neutres respectivement pour * et . On dit que f est un <u>morphisme de monoïde</u> de E dans F si et seulement si :

- f est un morphisme.
- f (e) = e'

Fin